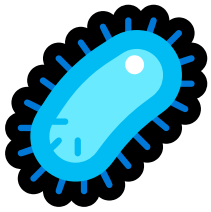

ম্যালওয়্যার বিশ্লেষণ ও রিভার্স ইঞ্জিনিয়ারিং — সম্পূর্ণ বাংলা গাইড

Static ও Dynamic Analysis থেকে শুরু করে Disassembly, Debugging, Unpacking, YARA Rules — Malware Analysis-এর সব কিছু এক জায়গায়।

📋 বিষয়সূচি

০১ Malware পরিচিতি ও ধরন

Virus, Worm, Trojan, Ransomware, RAT, Rootkit, Spyware

০২ Safe Lab Setup

Isolated VM, REMnux, FlareVM, Snapshot

০৩ Static Analysis

file, strings, hexdump, PE headers, imports, exports

PEiD, Detect-It-Easy, CFF Explorer, pestudio

০৪ Dynamic Analysis

Process Monitor, Wireshark, Regshot, FakeNet-NG

API call monitoring, Network traffic analysis

০৫ Sandbox Analysis

Any.run, Cuckoo Sandbox, Hybrid Analysis, VirusTotal

০৬ Reverse Engineering — Disassembly

Ghidra, IDA Pro, x64dbg, Binary Ninja

Assembly basics, Function analysis, Control flow

০৭ Packing ও Obfuscation

UPX, Custom packers, Anti-analysis techniques

String obfuscation, API hashing, Anti-debugging

০৮ YARA Rules

YARA rule লেখা, Test করা, Threat hunting

০৯ Malware পরিবার বিশ্লেষণ

Emotet, WannaCry, Cobalt Strike, Mimikatz

১০ Cheat Sheet & Roadmap

সব গুরুত্বপূর্ণ কমান্ড ও শেখার পথ

CHAPTER 01

Malware পরিচিতি ও ধরন

Malicious Software-এর সব ধরন ও তাদের কাজের পদ্ধতি

Malware কী?

Malware (Malicious Software) হলো যেকোনো সফটওয়্যার যা কোনো সিস্টেমের ক্ষতি করে, অনুমতি ছাড়া ডেটা চুরি করে, বা সিস্টেমের নিয়ন্ত্রণ নেয়। Malware Analyst-রা এই সফটওয়্যার বিশ্লেষণ করে বোঝেন এটা কী করে, কীভাবে ছড়ায় এবং কীভাবে রোধ করা যায়।

Malware-এর ধরন

Virus

নিজেকে অন্য ফাইলে inject করে ছড়ায়। Host file দরকার।

Self-replicating File infector

Worm

নেটওয়ার্কে নিজে নিজে ছড়ায়, Host file লাগে না। WannaCry এটা।

Self-propagating Network

Trojan

বৈধ সফটওয়্যার সেজে আসে, ভেতরে malicious code। Backdoor খোলে।

Disguised Backdoor

Ransomware

ফাইল encrypt করে মুক্তিপণ চায়। WannaCry, LockBit, REvil বিখ্যাত।

Encryption Ransom

RAT

Remote Access Trojan — পূর্ণ নিয়ন্ত্রণ দেয় attacker-কে। Keylog, Screenshot।

Remote Control C2

Rootkit

OS-এর গভীরে লুকিয়ে থাকে, নিজেকে hide করে। Detection কঠিন।

Stealth Kernel level

Keylogger

Keyboard-এর সব keystroke record করে পাঠায়। Password চুরি।

Keystroke Credential theft

🤖

Botnet/Bot

Infected machines C2 server-এর command follow করে। DDoS, spam।

C2 server DDoS

📊

Spyware/Infostealer

Browser password, cookie, credit card চুরি করে। Redline, Raccoon।

Data theft Stealer

Malware Analysis-এর লক্ষ্য

🎯 Analyst কী জানতে চায়?

- Malware কী কী করে? (Capability)
- C2 server কোথায়? (IOC)
- কীভাবে ছড়িয়েছে? (Infection vector)
- কোন ফাইল/Registry বদলেছে?
- কীভাবে Persistence রক্ষা করে?
- কোন family-র malware?

⚠️ IOC — Indicator of Compromise

- IP address / Domain (C2 server)
- File hash (MD5, SHA-256)
- File path, Registry key

- Mutex name
- Network signature
- Dropped files
- User-Agent string
- YARA rule match

💎 মনে রাখো

- ✅ Ransomware → File encrypt করে | RAT → Remote control | Rootkit → নিজেকে hide করে
- ✅ IOC → IP, Domain, Hash, File path — এগুলো খোঁজো
- ✅ Malware বিশ্লেষণ করো, নিজে তৈরি করো না — শুধু শিক্ষামূলক উদ্দেশ্যে

CHAPTER 02

🔬 Safe Lab Setup

Malware বিশ্লেষণের জন্য নিরাপদ isolated environment তৈরি

⚠️ সতর্কতা — প্রথমে পড়ো!

Real malware কখনো তোমার main machine-এ চালাবে না! সবসময় isolated VM-এ চালাবে — Network বন্ধ করে, Snapshot নিয়ে। একটি ভুলে তোমার পুরো সিস্টেম infected হতে পারে।

আদর্শ Lab Architecture

```
⚠️  ISOLATED NETWORK — Internet থেকে বিচ্ছিন্ন!

┌──────────────────────────────────────────────┐
│              Host Machine (তোমার PC)          │
│                                              │
│  ┌────────────────┐    ┌──────────────────┐  │
│  │ REMnux / Kali  │    │   Windows 10 VM  │  │
│  │ (Analyst)      │◄──►│   (Victim/Run)   │  │
│  │                │    │   FlareVM tools  │  │
│  │ Wireshark      │    │   Fakenet-NG     │  │
│  │ INetSim        │    │   Process Monitor│  │
│  │ FakeNet-NG     │    │   Regshot        │  │
│  └────────────────┘    └──────────────────┘  │
│                                              │
│  🔴 Host-only / Internal Network (No Internet!) │
└──────────────────────────────────────────────┘

Snapshot নাও → Malware চালাও → বিশ্লেষণ করো → Revert!
```

REMnux — Linux Analysis VM

```
# REMnux: https://remnux.org — Free, VMware/VirtualBox # Pre-installed tools: # Static: strings, file, hexdump, binwalk, upx, pefile # Dynamic: strace, ltrace, inetsim, fakedns # Network: Wireshark, tcpdump, NetworkMiner # RE: Ghidra, Cutter, radare2 # Memory: Volatility, rekall # Setup করো: # 1. VirtualBox/VMware-এ import করো # 2. Network → Host-only Adapter # 3. Snapshot নাও remnux upgrade # সব tool update
```

FlareVM — Windows Analysis VM

```
# FlareVM (Mandiant): Windows 10-এ install করো # https://github.com/mandiant/flare-vm # PowerShell (Admin)-এ: Set-ExecutionPolicy Unrestricted iex ((New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/mandiant/flare-vm/main/install.ps1')) # Pre-installed করে দেয়: # x64dbg, OllyDbg — Debugger # Ghidra, IDA Free — Disassembler # pestudio, CFF Explorer — PE Analysis # Process Hacker, Process Monitor — Process # Wireshark, FakeNet-NG — Network # HxD — Hex editor # Python, pefile — Scripting
```

INetSim — Fake Internet Simulator

```
# REMnux-এ INetSim চালাও — Malware-এর network call intercept করো # Fake HTTP, DNS, SMTP সার্ভার চালায় # /etc/inetsim/inetsim.conf: service_bind_address 0.0.0.0 dns_default_ip 192.168.56.101 # REMnux-এর IP start_service dns start_service http start_service https start_service smtp inetsim # চালু করো # Windows VM-এ DNS → REMnux IP দাও # এখন Malware যা-ই connect করুক → REMnux-এ যাবে # Wireshark দিয়ে দেখো: sudo wireshark -i eth0 &
```

🔬 Lab Setup মনে রাখো

- ✅ সবসময় Snapshot নিয়ে তারপর Malware চালাও
- ✅ Network → Host-only — Internet বন্ধ!
- ✅ REMnux = Linux analysis | FlareVM = Windows analysis
- ✅ INetSim → Fake internet, network call দেখো

CHAPTER 03

🔬 Static Analysis

Malware না চালিয়েই বিশ্লেষণ — নিরাপদ প্রথম পদক্ষেপ

Static Analysis কী?

Malware execute না করেই বিশ্লেষণ করা। File structure, strings, imports দেখে অনেক তথ্য পাওয়া যায়। এটা সবসময় প্রথম করো — নিরাপদ এবং দ্রুত।

Basic File Analysis — প্রথম ধাপ

```
═══════ File Type Identification ═══════ file suspicious.exe # File type (magic bytes দেখে) file -b suspicious.exe # Brief output # → PE32 executable (GUI) Intel 80386, UPX compressed ═══════ Hash — আসল পরিচয় ═══════ md5sum suspicious.exe # MD5 sha256sum suspicious.exe # SHA-256 (সবচেয়ে গুরুত্বপূর্ণ) sha1sum suspicious.exe # SHA-1 # Hash দিয়ে VirusTotal-এ সার্চ: curl "https://www.virustotal.com/api/v3/files/HASH" \ -H "x-apikey: YOUR_API_KEY" ═══════ Strings — Hidden Information ═══════ strings suspicious.exe # সব ASCII strings strings -n 8 suspicious.exe # Min 8 char strings suspicious.exe | grep -i "http\|ftp\|cmd\|power" strings suspicious.exe | grep -i "pass\|login\|key\|secret" strings suspicious.exe | grep -E "[0-9]{1,3}\.[0-9]{1,3}" # IP address strings suspicious.exe | grep -i "HKEY\|SOFTWARE\|Run" # Registry ═══════ Hex Dump ═══════ xxd suspicious.exe | head -30 # Hex + ASCII hexdump -C suspicious.exe | head -20 # PE file → MZ (4D 5A) দিয়ে শুরু # ELF file → 7F 45 4C 46 দিয়ে শুরু ═══════ Entropy — Packing/Encryption ═══════ # High entropy (7.0+) → Packed বা Encrypted! python3 -c " import math, collections data = open('suspicious.exe','rb').read() cnt = collections.Counter(data) entropy = -sum(v/len(data)*math.log2(v/len(data)) for v in cnt.values() if v) print(f'Entropy: {entropy:.2f}') "
```

PE File Analysis — Windows Executable

```
# PE (Portable Executable) Structure: # MZ Header → PE Header → Section Headers → Sections # Python pefile দিয়ে: import pefile pe = pefile.PE('suspicious.exe') # Compile time: import datetime print(datetime.datetime.fromtimestamp(pe.FILE_HEADER.TimeDateStamp)) # Sections দেখো (entropy চেক করো): for section in pe.sections: print(section.Name, hex(section.VirtualAddress), section.SizeOfRawData, section.get_entropy()) # Imports — কোন Windows API call করে? for entry in pe.DIRECTORY_ENTRY_IMPORT: print(entry.dll.decode()) for imp in entry.imports: print(' ', imp.name) # Suspicious imports: CreateRemoteThread # → Code injection! VirtualAllocEx # → Inject করার আগে memory allocate WriteProcessMemory # → অন্য process-এ write WinExec / ShellExecute # → Program চালানো URLDownloadToFile # → Internet থেকে download RegSetValueEx # → Registry বদলানো (Persistence!) CryptEncrypt # → Ransomware!
```

pestudio — GUI Static Analysis

```
# pestudio (Windows) — সবচেয়ে সহজ PE analysis tool # Drag & drop করো → সব দেখাবে # দেখবে: # ✓ Virus Total score # ✓ Imports (highlighted suspicious ones) # ✓ Strings (URLs, IPs, Registry keys) # ✓ Sections ও entropy # ✓ Certificates # ✓ Resources (hidden files?) # Detect-It-Easy (DIE) — Packer detect: die suspicious.exe # → UPX (3.95) [Packer] # → .NET(v4.0.30319) [Linker]
```

Suspicious Strings — কী দেখলে সতর্ক হবে?

String	সম্ভাব্য মানে	ধরন
`http://` , `https://`	C2 server বা download URL	HIGH
`cmd.exe` , `powershell`	Shell execution	HIGH
`HKEY_` , `\Run\`	Registry persistence	CRITICAL

String	সম্ভাব্য মানে	ধরন
`encrypt` , `ransom`	Ransomware	CRITICAL
`bitcoin` , `.onion`	Ransom payment	CRITICAL
`keylog` , `screenshot`	Spyware/RAT	HIGH
`VirtualAlloc` , `CreateRemoteThread`	Code injection	CRITICAL
IP address pattern	Hardcoded C2	HIGH

🔬 Static Analysis মনে রাখো

- ✅ `file` → file type | `strings` → hidden text | `sha256sum` → hash
- ✅ High entropy (7+) → packed বা encrypted
- ✅ Imports: `CreateRemoteThread` , `URLDownloadToFile` → বিপজ্জনক
- ✅ pestudio + DIE = Windows-এ সবচেয়ে সহজ tool

CHAPTER 04

🏃 Dynamic Analysis

Malware চালিয়ে behavior পর্যবেক্ষণ — VM-এ, Snapshot নিয়ে!

Dynamic Analysis কী?

Malware সত্যিই চালিয়ে তার behavior monitor করা। Static Analysis-এ যা বোঝা যায় না — packed বা obfuscated malware — সেগুলো Dynamic Analysis-এ ধরা যায়।

Process Monitor (Procmon) — সব কিছু দেখো

```
# Process Monitor (Sysinternals) — Windows-এ সেরা dynamic tool # File, Registry, Network, Process সব কার্যক্রম দেখায় # Filter করো (malware process শুধু): # Filter → Process Name → is → malware.exe → Include # দেখবে: # File Created: C:\Users\Public\backdoor.exe ← dropped file! # Registry SetValue: HKCU\Software\Microsoft\Windows\CurrentVersion\Run ← Persistence! # TCP Connect: 192.168.1.100:4444 ← C2 connection! # Process Created: cmd.exe ← Shell! # Linux-এ: strace ও ltrace strace -f ./malware 2>&1 | tee strace_output.txt strace -f -e trace=network ./malware # শুধু network call strace -f -e trace=file ./malware # শুধু file operation ltrace ./malware # Library call
```

Regshot — Registry পরিবর্তন দেখো

```
# Regshot — Registry snapshot before/after malware # 1st Shot → Malware চালাও → 2nd Shot → Compare! # দেখতে পাবে কোন Registry key: # Added: HKCU\Software\Evil\C2Server = "http://evil.com" # Modified: HKLM\...\Run\malware = "C:\Windows\backdoor.exe" # Deleted: ... # Common Persistence Registry Keys: HKCU\Software\Microsoft\Windows\CurrentVersion\Run HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

```
HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\SYSTEM\CurrentControlSet\Services\ # Service install
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
```

Wireshark — Network Traffic Analysis

```
# Wireshark দিয়ে malware-এর network activity ধরো # Capture filter (malware VM শুধু): host 192.168.56.102 # VM-এর IP # Display filter: http # HTTP traffic dns # DNS query (C2 domain name!) tcp.port == 4444 # Metasploit default port ip.addr == 192.168.56.1 # Malware network IOC: # DNS query: evilc2server.ru → C2 domain! # HTTP POST /beacon → C2 beacon! # User-Agent: Mozilla/5.0 (unusual) → Hardcoded! # Large upload → Data exfiltration! # FakeNet-NG — REMnux-এ fakenet # Fake internet, log করে সব request # → Log দেখো: /var/log/fakenet/ # tcpdump — CLI-তে capture: sudo tcpdump -i eth0 -w malware_traffic.pcap sudo tcpdump -i eth0 'host 192.168.56.102' -w capture.pcap
```

Process Hacker / API Monitor

```
# Process Hacker — Process ও Memory দেখো # Malware-এর process সঠিক location-এ আছে কিনা? # svchost.exe কিন্তু C:\Users\-এ → Suspicious! # Parent process: Word.exe → cmd.exe → malware.exe (macro!) # Memory strings (runtime decryption ধরো): # Process Hacker → Process → Memory → Strings # Packed malware unpack হওয়ার পরে এখানে আসল strings দেখা যাবে! # API Monitor — কোন API call হচ্ছে? # Hook করে সব Windows API call দেখায় # Linux: ltrace, strace strace -p PID # চলমান process-এর syscall cat /proc/PID/maps # Memory map cat /proc/PID/net/tcp # Network connection ls -la /proc/PID/fd/ # Open files
```

🏃 Dynamic Analysis মনে রাখো

- ✅ Procmon → File/Registry/Network সব কার্যক্রম দেখো
- ✅ Regshot → Before/After Registry compare
- ✅ Wireshark → C2 domain, IP, beacon pattern ধরো

- ✅ Snapshot নিয়ে চালাও, Revert করো — কখনো ভুলো না!

CHAPTER 05

Sandbox Analysis

Online automated analysis — দ্রুত প্রথম তথ্য পাওয়ার পদ্ধতি

Online Sandbox — কখন ব্যবহার করবে?

⚠ **সতর্কতা**

Online sandbox-এ sample upload করলে সেটা public হয়ে যায়! Sensitive/confidential malware upload করো না। শুধু public malware বা CTF challenge-এ ব্যবহার করো।

সেরা Sandbox Platforms

VirusTotal

- **URL:** virustotal.com
- ৭০+ AV engine দিয়ে scan করে
- Hash দিয়ে search করো (upload না করে)
- Import, Strings, Behavior দেখায়
- Similar files, community comments
- Free

Any.run

- **URL:** any.run
- Interactive sandbox — real-time দেখো

- Process tree, Network, Registry
- MITRE ATT&CK mapping
- IOC extract করে
- **Free (public)** **Paid (private)**

🔬 Hybrid Analysis

- **URL:** hybrid-analysis.com
- Falcon Sandbox (CrowdStrike)
- YARA match, Suricata alerts
- Detailed report, IOC export
- **Free**

🦅 Cuckoo Sandbox

- **Self-hosted** — Private!
- নিজের server-এ চালাও
- Windows/Linux/Android support
- Network PCAP, Memory dump
- **Free, Open Source**

VirusTotal API — Automated Analysis

```
# Hash দিয়ে VT check (কমান্ড লাইনে): curl --request GET \ --url "https://www.virustotal.com/api/v3/files/HASH_HERE" \ --header "x-apikey: YOUR_API_KEY" # Python দিয়ে: import requests API_KEY = "your_api_key" file_hash = "d41d8cd98f00b204e9800998ecf8427e" url = f"https://www.virustotal.com/api/v3/files/{file_hash}" r = requests.get(url, headers={"x-apikey": API_KEY}) data = r.json() stats = data["data"]["attributes"]
```

```
["last_analysis_stats"] print(f"Malicious: {stats['malicious']}/{sum(stats.values())}") # Threat Intelligence Tools: # MalwareBazaar: bazaar.abuse.ch — Free malware samples # ThreatFox: threatfox.abuse.ch — IOC database # URLhaus: urlhaus.abuse.ch — Malicious URLs # OTX AlienVault: otx.alienvault.com — Threat intel
```

🛡️ Sandbox মনে রাখো

- ✅ প্রথমে Hash দিয়ে VT-তে search করো — upload করো না
- ✅ Any.run → Interactive, real-time দেখো
- ✅ Cuckoo → Private/sensitive sample-এর জন্য self-hosted
- ✅ MalwareBazaar → Real malware sample practice-এর জন্য

CHAPTER 06

Reverse Engineering — Disassembly

Binary থেকে source code বের করার কৌশল — Ghidra ও x64dbg

Assembly মূল ধারণা — RE-তে যা জানতে হবে

```
# x86 Registers (32-bit): EAX # Accumulator — return value, math EBX # Base — general purpose ECX # Counter — loop counter EDX # Data — general purpose ESP # Stack Pointer — stack top EBP # Base Pointer — function frame ESI/EDI # Source/Destination — string ops EIP # Instruction Pointer — next instruction # x64: RAX, RBX, RCX... (64-bit) # Common Instructions: MOV EAX, 1 # EAX = 1 PUSH EAX # Stack-এ push POP EBX # Stack থেকে pop CALL func # Function call RET # Return JMP addr # Unconditional jump JE / JZ # Jump if equal/zero JNE / JNZ # Jump if not equal CMP EAX, EBX # Compare (flags set করে) XOR EAX, EAX # EAX = 0 (zero out) LEA EAX, [EBP-4] # Load Effective Address NOP # No operation (0x90)
```

Ghidra — Free Disassembler (NSA)

```
# Ghidra: https://ghidra-sre.org — Free, NSA developed # Java required # Workflow: # 1. New Project → Import File (drag & drop) # 2. Analysis চালাও (Auto Analysis) # 3. Symbol Tree → Functions → main খোঁজো # 4. Decompiler window দেখো (C-like code!) # Ghidra-তে কী দেখবে: # Left panel: Symbol Tree (functions, imports) # Middle: Disassembly (Assembly code) # Right: Decompiler (C-like pseudocode) ← সবচেয়ে গুরুত্বপূর্ণ # Script দিয়ে automate: # Ghidra → Window → Script Manager → Python # Command line (headless): ./analyzeHeadless /project proj_name -import malware.exe \ -postScript PrintStrings.py
```

x64dbg — Windows Debugger

```
# x64dbg: https://x64dbg.com — Free Windows debugger # Important Shortcuts: F2 # Breakpoint set/remove F7 # Step Into (function-এ ঢোকো) F8 # Step Over (function skip করো) F9 # Run (next breakpoint পর্যন্ত) F4 # Run to cursor Ctrl+G # Go to address # Common Breakpoints: # bp CreateFileA → ফাইল তৈরির আগে থামো # bp URLDownloadToFileA → Download-এর আগে # bp RegSetValueExA → Registry write-এর আগে # bp VirtualAlloc → Memory allocation # bp WSASend/send → Network send # API call-এ breakpoint: # Plugins → APIBreak → Add API name # Memory dump (unpacked malware বের করো): # OEP (Original Entry Point) খোঁজো → Dump → Rebuild PE # Scylla plugin → Dump → Fix Imports → Save
```

radare2 / Cutter — Linux RE

```
# radare2 — CLI disassembler r2 suspicious.elf # Open aaa # Analyze all afl # List functions pdf @ main # Disassemble main VV # Visual/graph mode s main; pdf # Seek to main, print iz # Strings ii # Imports q # Quit # Cutter = radare2 GUI version cutter suspicious.elf # objdump (basic, pre-installed): objdump -d suspicious.elf | head -50 objdump -t suspicious.elf # Symbol table objdump -x suspicious.elf # Headers # GDB — Linux debugger: gdb ./suspicious break main run disas main info registers x/10x $esp # Stack দেখো
```

⚙️ RE মনে রাখো

- ✅ Ghidra → Decompiler = C-like code, বোঝা সহজ
- ✅ x64dbg → F2 Breakpoint, F7 Step, F9 Run
- ✅ `EAX` = return value | `ESP` = stack top | `EIP` = next instruction
- ✅ `JE/JNE` → condition check — license check bypass এখানে

CHAPTER 07

📦 Packing & Obfuscation

Malware কীভাবে নিজেকে লুকায় — এবং কীভাবে বের করবে

Packing কী?

Packer হলো একটি tool যা executable-কে compress বা encrypt করে, যাতে AV detection কঠিন হয়। Runtime-এ নিজেকে memory-তে unpack করে চলে। সবচেয়ে সাধারণ packer হলো UPX।

UPX Unpacking

```
# UPX detect করো: die suspicious.exe # → UPX(3.96) [Packer] strings suspicious.exe | grep UPX # UPX! string থাকে # UPX unpack (সহজ): upx -d suspicious.exe -o unpacked.exe upx -d suspicious.exe # In-place # Verify: file unpacked.exe strings unpacked.exe | head -50 # এখন আসল strings দেখা যাবে # Custom packer (UPX নয়) → Manual unpacking দরকার: # 1. x64dbg-তে open করো # 2. ESP trick: Entry point-এ PUSHAD → VirtualAlloc → copy → POPAD → JMP OEP # 3. OEP (Original Entry Point) খোঁজো # 4. Scylla দিয়ে dump করো # Memory dump করো (x64dbg): # OEP-এ breakpoint → run → Memory Map → Right-click → Dump
```

Anti-Analysis Techniques — Malware কীভাবে বাধা দেয়

কৌশল	কীভাবে কাজ করে	কীভাবে bypass করবে
Anti-Debugging	`IsDebuggerPresent()` check করে	x64dbg ScyllaHide plugin
VM Detection	VM artifact check করে (VMware registry)	VM artifact মুছো, patch করো
Timing Check	`GetTickCount()` দিয়ে slow execution detect	Patch the check, NOP it

কৌশল	কীভাবে কাজ করে	কীভাবে bypass করবে
String Obfuscation	XOR/Base64 দিয়ে strings encode	Dynamic analysis, deobfuscate
API Hashing	API name hash দিয়ে call করে, string নেই	HashDB plugin (IDA/Ghidra)
Process Injection	অন্য process-এ code inject	Process Hacker, breakpoint

String Obfuscation — Decode করো

```
# XOR obfuscation — সবচেয়ে সাধারণ # Malware: XOR key দিয়ে strings encode করে রাখে # Runtime-এ decode করে ব্যবহার করে # Python দিয়ে XOR decode: encoded = [0x48, 0x65, 0x6c, 0x6c, 0x6f] # Example key = 0x05 decoded = bytes([b ^ key for b in encoded]) print(decoded.decode()) # xortool: xortool suspicious.bin # XOR key guess করে xortool -x -k 05 file.bin # Key 0x05 দিয়ে XOR # CyberChef (Web UI): # XOR → Base64 → ROT13 → chain করে decode # Base64 encoded strings: echo "SGVsbG8=" | base64 -d python3 -c "import base64; print(base64.b64decode('SGVsbG8='))"
```

📦 Packing মনে রাখো

- ✅ UPX → `upx -d file.exe` দিয়ে unpack
- ✅ High entropy → packed | strings নেই → obfuscated
- ✅ Anti-debug → ScyllaHide plugin ব্যবহার করো
- ✅ XOR decode → xortool বা CyberChef

CHAPTER 08

🔎 YARA Rules

Malware চেনার Pattern Matching — Threat Hunting-এর মূল অস্ত্র

YARA কী?

YARA হলো malware researcher-দের জন্য pattern matching tool। বিশেষ string, hex pattern বা condition দিয়ে malware চেনা যায়। AV, SIEM, Threat Intel platform-এ ব্যাপকভাবে ব্যবহৃত।

YARA Rule লেখার পদ্ধতি

```
# Basic YARA Rule Structure: rule RuleName { meta: description = "কী detect করবে" author = "তোমার নাম" date = "2025-01-01" severity = "high" strings: $s1 = "http://c2server.evil" # ASCII string $s2 = "password.exe" nocase # Case insensitive $s3 = "ransom" wide # Unicode (wide char) $h1 = { 4D 5A 90 00 03 00 00 00 } # Hex pattern (PE header) $h2 = { 60 ?? ?? ?? 61 } # ?? = wildcard byte $r1 = /https?:\/\/[a-z0-9]+\.ru\// # Regex condition: # সব string থাকলে match: all of them # অথবা যেকোনো একটি থাকলে: any of ($s*) # File size check সহ: filesize < 1MB and $s1 and $h1 # PE file হতে হবে: uint16(0) == 0x5A4D and $s1 # MZ header }
```

Real YARA Rules উদাহরণ

```
# Ransomware detect rule: rule Ransomware_Generic { meta: description = "Generic Ransomware Detection" severity = "critical" strings: $ransom1 = "Your files have been encrypted" nocase $ransom2 = "bitcoin" nocase $ransom3 = "decrypt" nocase $ransom4 = ".onion" $ransom5 = "pay" nocase $crypto1 = "CryptEncrypt" $crypto2 = "CryptoAPI" condition: 3 of ($ransom*) or (1 of ($ransom*) and 1 of ($crypto*)) }
# Suspicious PE with download capability: rule Suspicious_Downloader { meta: description = "PE file that downloads content" strings: $url = /https?:\/\/[0-9]
```

```
{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}/ $api1 = "URLDownloadToFile" $api2 = "WinINet" $api3 = "HttpSendRequest" condition: uint16(0) == 0x5A4D and filesize < 5MB and $url and any of ($api*) }
```

YARA চালানো

```
# Install: sudo apt install yara # Linux pip install yara-python # Python # একটি file scan: yara my_rule.yar suspicious.exe # Directory scan: yara -r my_rule.yar /path/to/files/ # Multiple rules: yara rules/*.yar suspicious.exe # Python দিয়ে: import yara rules = yara.compile('my_rule.yar') matches = rules.match('suspicious.exe') for match in matches: print(match.rule, match.strings) # Online YARA resources: # Yara Rules GitHub: https://github.com/Yara-Rules/rules # Awesome YARA: https://github.com/InQuest/awesome-yara # yarahub.io — community YARA rules
```

🔎 YARA মনে রাখো

- ✅ String + Hex + Regex → তিন ধরনের pattern
- ✅ `uint16(0) == 0x5A4D` → PE file চেনার উপায়
- ✅ `nocase` , `wide` → Case ও Unicode handle করে
- ✅ Real rules: Yara-Rules GitHub থেকে নাও

CHAPTER 09

🌍 বিখ্যাত Malware পরিবার

Real-world malware-এর বিশ্লেষণ — ইতিহাস থেকে শিক্ষা

WannaCry — বিশ্বের সবচেয়ে বিখ্যাত Ransomware

💀 WannaCry (2017)

- Type: Ransomware Worm
- Exploit: EternalBlue (NSA, CVE-2017-0144)
- Protocol: SMB v1 (Port 445)
- Extension: .WNCRY
- Ransom: $300-600 Bitcoin
- Kill switch: iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com
- ১৫০+ দেশে ২,৩০,০০০+ system আক্রান্ত

🔬 IOC

- MD5: `db349b97c37d22f5ea1d1841e3c89eb4`
- Dropped: `tasksche.exe`
- Registry: `HKCU\SOFTWARE\WanaCrypt0r`
- Mutex: `Global\MsWinZonesCacheCounterMutexA`
- Bitcoin wallets (3টি)
- AES + RSA encryption
- Ransom note: @Please_Read_Me@.txt

Emotet — সবচেয়ে বিপজ্জনক Botnet

```
# Emotet — Modular Banking Trojan / Loader (2014-2021, 2022-) # ★ Infection vector: Malicious Word/Excel macro # ★ Payload: Download and execute (TrickBot, Ryuk) # Infection chain: # Phishing email → Word doc → Enable Macros → PowerShell → Download Emotet # Emotet → Spread via email (steals contacts) → Download TrickBot # TrickBot → Credential theft → Domain admin → Ryuk ransomware # Emotet IOC: # Network: HTTPS beacon to C2 (encrypted) # Process: regsvr32.exe, wscript.exe, powershell.exe # Registry: HKCU\Software\Microsoft\Windows\CurrentVersion\Run # Persistence: Scheduled Task বা Registry Run key # Detection: strings emotet_sample.exe | grep -i "powershell\|wscript\|regsvr"
```

Cobalt Strike — Red Team Tool → Malware হিসেবে

```
# Cobalt Strike — Legitimate pentest tool, APT groups ব্যবহার করে # Beacon → C2 communication # Cobalt Strike Beacon detect করো: strings beacon.exe | grep -i "sleep\|checkin\|pipe" # Network pattern: # HTTP/HTTPS C2 polling (configurable interval) # User-Agent: hardcoded Mozilla string # 1768.py — CS config extract: python3 1768.py -r beacon.exe # Shodan dork (exposed CS servers): # product:"Cobalt Strike Beacon" # YARA rule for CS: # strings: "MZ" PE header + specific CS strings # https://github.com/Neo23x0/signature-base
```

Mimikatz — Credential Theft Tool

```
# Mimikatz — Windows credential extraction (Post-exploitation) # LSASS memory থেকে plaintext password বের করে # Common commands (Attacker দৃষ্টিকোণ থেকে বোঝার জন্য): privilege::debug # Debug privilege পাওয়া sekurlsa::logonpasswords # Plaintext password ও hash বের করা sekurlsa::pth # Pass-the-Hash attack lsadump::sam # SAM database dump lsadump::dcsync # Domain Controller sync (AD attack) kerberos::golden # Golden ticket attack # Mimikatz detect করো: strings mimikatz.exe | grep -i "sekurlsa\|lsass\|wdigest" # Defense: # Windows Defender
```

```
Credential Guard চালু করো # Protected Users group ব্যবহার করো # Disable WDigest:
HKLM\SYSTEM\...\WDigest\UseLogonCredential = 0
```

🌍 Famous Malware মনে রাখো

- ✅ WannaCry → EternalBlue SMB exploit → AES+RSA encryption
- ✅ Emotet → Macro → PowerShell → Loader for other malware
- ✅ Cobalt Strike → Legitimate tool, APT groups ব্যবহার করে
- ✅ Mimikatz → LSASS → Plaintext password বের করে

CHAPTER 10

📋 Cheat Sheet & Roadmap

সব গুরুত্বপূর্ণ কমান্ড ও Malware Analyst হওয়ার পথ

🔬 Static Analysis Cheat Sheet

📂 File Info

`file suspicious.exe`	File type
`sha256sum file.exe`	Hash (VT search)
`strings -n 8 file.exe`	Hidden strings
`xxd file.exe | head`	Hex dump
`die file.exe`	Packer detect
`upx -d file.exe`	UPX unpack

🐍 PE Analysis

`python3 -c "import pefile; pe=pefile.PE('f.exe'); [print(e.dll) for e in pe.DIRECTORY_ENTRY_IMPORT]"`	Imports
`pestudio → open file`	GUI PE analysis
`objdump -d file.exe`	Disassemble
`readelf -a file.elf`	ELF headers

🏃 Dynamic Analysis Cheat Sheet

🐧 Linux (strace)

`strace -f ./malware 2>&1 | tee log.txt`	All syscalls
`strace -e trace=network ./m`	Network only
`ltrace ./malware`	Library calls
`tcpdump -i eth0 -w cap.pcap`	Network capture
`cat /proc/PID/maps`	Memory map

Windows Tools

`procmon.exe → Filter: process`	File/Reg/Net monitor
`Regshot → 1st shot → run → 2nd → compare`	Registry diff
`wireshark -i eth0`	Network traffic
`fakenet`	Fake internet
`processhacker.exe`	Process & memory

⚙️ RE & YARA Cheat Sheet

⚙️ Reverse Engineering

`ghidra → open → analyze → decompile`	Free disassembler
`r2 file; aaa; afl; pdf @ main`	radare2
`gdb -q ./elf`	Linux debug
`x64dbg → F2 BP, F7 step, F9 run`	Windows debug
`upx -d packed.exe`	UPX unpack

🔎 YARA

`yara rule.yar suspect.exe`	Single scan
`yara -r rules/ /path/`	Directory scan
`uint16(0) == 0x5A4D`	PE file check
`$s = "evil" nocase wide`	String match

`any.run → upload → IOC download` Sandbox

🗺️ Malware Analyst Roadmap

1

📚 Phase 1 — ভিত্তি 1-2 মাস

OS internals, networking, basic programming না জানলে malware বোঝা কঠিন।

Windows Internals | Linux basics | Python scripting | C/C++ basics | Assembly x86

↓

2

🔬 Phase 2 — Static Analysis 1-2 মাস

File analysis, PE structure, strings — malware না চালিয়ে বিশ্লেষণ শেখো।

REMnux setup | file/strings/hexdump | pefile Python | pestudio, DIE | VirusTotal

↓

3

🏃 Phase 3 — Dynamic Analysis 1-2 মাস

VM setup করো, sandbox ব্যবহার শেখো, behavior monitor করো।

FlareVM setup | Procmon, Regshot | Wireshark | Any.run | Cuckoo Sandbox

↓

4

⚙️ Phase 4 — Reverse Engineering 2-3 মাস

Assembly, Ghidra, Debugger — এখানেই সত্যিকারের দক্ষতা তৈরি হয়।

x86 Assembly | Ghidra/IDA | x64dbg | YARA rules | CTF Reversing

↓

5

🏆 Phase 5 — Real Malware ও Career চলমান

Real malware বিশ্লেষণ করো, writeup লেখো, community-তে contribute করো।

MalwareBazaar samples | Write reports | GREM cert | Threat Intel | SOC Analyst

📚 সেরা Resources

Resource	ধরন	বিষয়
Practical Malware Analysis (book)	Book	সেরা বই — Sikorski & Honig
MalwareBazaar	Samples	Free real malware samples
Any.run	Sandbox	Interactive analysis
OpenSecurityTraining2	Course	Free RE/malware courses
Ghidra Course (NSA)	Course	Official Ghidra training
CTF RE challenges	Practice	picoCTF, crackmes.one
REMnux	VM	Free analysis distro
Yara-Rules GitHub	Rules	Community YARA rules

🔬 "Know your enemy — analyze the malware before it spreads."

এই ডকুমেন্ট সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। Malware বিশ্লেষণ করো, তৈরি বা ছড়িও না।

v1.0 — Malware Analysis & Reverse Engineering | Bilingual | Dark/Light Mode